头指针

```
头指针是只想链表第一个结点的指针
头指针具有标识作用，常用来冠以链表的名字
```

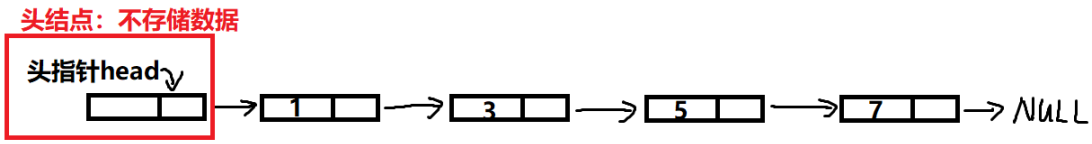
尾指针

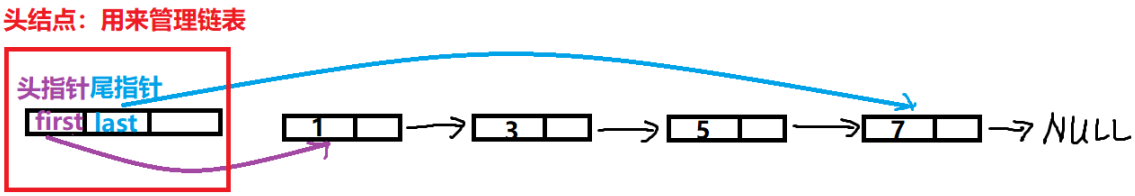
```
尾指针是指向链表最后一个结点的指针
```

头结点

```
模型1
头结点的类型和数据结点类型一致，但是头结点不存储数据(有些地方会存储数据元素的个数)
头指针则为指向头结点的指针
作用：有了头结点，对第一个数据结点进行插入和删除操作
这种头结点也会被称为哑结点
```

```
模型2
    专门用于管理和标识这个链表的结点，不是链表中的必要元素
    找第一个结点和最后一个结点，获取数据结点的个数会更方便
    struct node//数据结点
    {
        int data;
        struct node *next;
    };
    struct list//管理
    {
        struct node *first;//指定链表中第一个结点
        struct node *last;//指定链表中最后一个结点
        int num;//保存链表中数据结点的个数
        ......
    }
```





```
//类型的定义
typedef int elemtype;
typedef struct node
{
    elemtype data;//数据域
    struct node *next;
}node;
```

注意：找的时候都从head->next开始找

1）更具用户的输入顺序，创建一个带头结点的单链表，用模型1，打印输出

linklist_with_head.c/linklist_with_head.h/main.c

```
node *create_list_with_head()
{
    //1.创建一个头结点
    node *head = malloc(sizeof(node));
    head->data = 0;//存储结点的数目
    head->next = NULL;

    while(1)
    {
        //输入数据

        //创建结点

        //尾插法
    }
}
```

2）创建带头结点的升序单链表，打印输出(只有中间插和尾插)

3) 找到值为x的结点，将其全部进行删除，若没有找到，就不删除，并且将新链表进行返回 (只有中间删)

4) 销毁单链表(头结点也需要销毁，就是循环头删)

5) 在head指向的单链表中，结点的值有正数，也有负数，想办法调整该链表，使得负数在正数的前面，并且将新链表返回，要求：不能排序，不能额外申请结点的空间